Multi-Modal Deep Learning for Fashion Product Classification

Theoretical & Mathematical Report

Course: Deep Learning · **Team:** Person A (image/CNN), Person B (text/RNN), Person C (data, fusion, infrastructure)

Abstract

We build a multi-modal classifier that predicts a fashion product's `subCategory` from **both** its product image and its short text description (`productDisplayName`). The system has three parts — a convolutional image encoder (a frozen EfficientNetB0 backbone), a recurrent text encoder (trainable embedding → GRU), and a fusion head that concatenates the two encodings and classifies. To demonstrate that multi-modal fusion is worthwhile, we train and evaluate **three** models on the identical data pipeline: image-only, text-only, and the fused model. This report derives the mathematics of every component, states each design choice and why it was made, and reports the three-way comparison that is the project's central result.

---

1. Problem statement and data

We use the Kaggle *Fashion Product Images (small)* dataset. Each product has an image (`images/{id}.jpg`) and a metadata row in `styles.csv` . We classify the `subCategory` field, restricted to the **top-10 most frequent classes**, using the `productDisplayName` string as the text modality.

Formally, for a product i with image X_i and text t_i, we learn a function

$$f_\theta : (X_i, t_i) \longmapsto \hat{y}_i \in \Delta^{K-1}, \qquad K = 10,$$

where Δ^{K-1} is the probability simplex over the K classes, and predict $\arg\max_k \hat{y}_{i,k}$.

Why this target? `masterCategory` has only a handful of broad buckets — almost perfectly separable from the image alone, so fusion would add nothing observable. `articleType` has 140+ classes — too hard for the timeline. The top-10 `subCategory` set is the regime where the **marginal value of the second modality is measurable**, which is exactly what the assignment asks us to show.

The dataset is split **stratified 70/15/15** into train/validation/test with a fixed seed (§7). All three models use this identical split and the identical text vectoriser, so differences in their metrics are attributable only to the model, not to data handling.

---

2. Image branch — Convolutional Neural Network

2.1 The convolution operation

A convolutional layer slides learnable kernels over the spatial dimensions of its input. Let the input feature map be $X \in \mathrm{R}^{H \times W \times C}$ (height, width, channels) and let a layer have F filters, each a tensor $K^{(f)} \in \mathrm{R}^{k_h \times k_w \times C}$ with bias b_f. The pre-activation output at spatial position (i, j) and filter f is

$$Z_{i,j,f} = b_f + \sum_{c=1}^{C} \sum_{u=1}^{k_h} \sum_{v=1}^{k_w} K^{(f)}_{u,v,c} X_{i \cdot s+u-1,\, j \cdot s+v-1,\, c},$$

where s is the stride. With padding p, the output spatial size is

$$H_{\text{out}} = \lfloor \frac{H - k_h + 2p}{s} \rfloor + 1, \qquad W_{\text{out}} = \lfloor \frac{W - k_w + 2p}{s} \rfloor + 1.$$

Two properties make convolution the right inductive bias for images:

- **Local connectivity** — each output depends only on a small $k_h \times k_w$ receptive field, matching the locality of visual structure (edges, textures).
- **Parameter sharing** — the *same* kernel is applied at every location, giving **translation equivariance**: shifting the input shifts the feature map. This drastically reduces parameters versus a fully connected layer ($k_h k_w CF$ weights instead of $HWC \cdot H_{\text{out}} W_{\text{out}} F$).

2.2 Activation functions

After the linear convolution we apply a pointwise nonlinearity; without it a stack of convolutions would collapse into a single linear map. The classic choice is the **ReLU**:

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0. \end{cases}$$

ReLU is cheap, non-saturating for $z > 0$ (mitigating vanishing gradients), and induces sparsity. EfficientNet internally uses the smooth **Swish / SiLU** activation,

$$\mathrm{swish}(z) = z\,\sigma(z) = \frac{z}{1 + e^{-z}}, \qquad \mathrm{swish}'(z) = \sigma(z) + z\,\sigma(z)(1 - \sigma(z)),$$

which is differentiable everywhere and empirically improves deep CNN accuracy. Our own projection head uses ReLU (§2.4).

2.3 Pooling

Pooling downsamples a feature map, providing local translation **invariance** and shrinking compute. Over a pooling region R:

$$\text{max-pool: } Y_{i,j,c} = \max_{(u,v)\in \mathrm{R}} X_{u,v,c}, \qquad \text{avg-pool: } Y_{i,j,c} = \frac{1}{|\mathrm{R}|} \sum_{(u,v)\in \mathrm{R}} X_{u,v,c}.$$

We use **Global Average Pooling (GAP)** to turn the backbone's final feature map $X \in \mathrm{R}^{H' \times W' \times F}$ into one vector by averaging each channel over all spatial positions:

$$g_f = \frac{1}{H'W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} X_{i,j,f}, \qquad g \in \mathrm{R}^F.$$

GAP has **no parameters**, is robust to spatial shifts, and yields a compact fixed-length descriptor ideal for feeding a dense classifier or a fusion head.

2.4 Architecture and the EfficientNetB0 choice

$$\underbrace{\text{EfficientNetB0 (ImageNet, frozen)}}_{\text{feature extractor}} \rightarrow \text{GlobalAveragePooling2D} \rightarrow \text{Dense}(256,\ \text{ReLU}) =$$

Why EfficientNetB0? It is obtained by *compound scaling* — jointly scaling network depth $d = \alpha^\phi$, width $w = \beta^\phi$, and input resolution $r = \gamma^\phi$ under the constraint $\alpha\beta^2\gamma^2 \approx 2$ — which gives an excellent accuracy/FLOP trade-off. B0 is the smallest variant (~5.3M parameters), which suits Colab's free GPU.

Why freeze the backbone? Transfer learning: ImageNet features (edges → textures → parts → objects) transfer well to product photos. With a modest dataset, freezing (i) prevents overfitting the large backbone, (ii) cuts training cost to just the small head, and (iii) keeps BatchNorm layers in **inference mode** (using their stored running statistics rather than noisy batch statistics), which stabilises training. We optionally unfreeze the top blocks for low-LR fine-tuning only if time permits (stretch goal).

> *Limitation, documented:* the small dataset's images are ~60×80 px and are upscaled to EfficientNet's 224×224 input. This loses high-frequency detail; we accept it as a course-project trade-off and note it in the discussion.

---

3. Text branch — Embeddings + Recurrent Neural Network

3.1 Tokenisation and word embeddings

The raw string is lower-cased, stripped of punctuation, split into tokens, mapped to integer indices by a `TextVectorization` layer (vocabulary size V, padded/truncated to length L), and looked up in an embedding matrix $E \in \mathrm{R}^{V \times d}$:

$$t = (w_1, \ldots, w_L), \qquad e_t = E_{w_t,:} \in \mathrm{R}^d.$$

The embedding is **trainable** (learned end-to-end), so semantically similar tokens ("shirt", "tee") drift toward nearby vectors. We deliberately avoid GloVe/BERT: the descriptions are short and domain-specific, so a compact trainable embedding is lighter and sufficient (see scope cuts, §7). The vectoriser is

adapted on the training split only and persisted, so all three models share an identical vocabulary with no information leaking from validation/test.

3.2 The GRU recurrence

A Gated Recurrent Unit processes the embedding sequence $e_1, \dots, e_L$, maintaining a hidden state $h_t \in \mathrm{R}^n$. At each step (σ = logistic sigmoid, $\odot$ = elementwise product):

$$\begin{aligned} z_t &= \sigma(W_z e_t + U_z h_{t-1} + b_z) && \text{(update gate)} \\ r_t &= \sigma(W_r e_t + U_r h_{t-1} + b_r) && \text{(reset gate)} \\ \widetilde{h}_t &= \tanh(W_h e_t + U_h (r_t \odot h_{t-1}) + b_h) && \text{(candidate state)} \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \widetilde{h}_t. && \text{(new state)} \end{aligned}$$

Interpretation:

- The **reset gate** r_t decides how much of the previous state to *forget* when forming the candidate $\widetilde{h}_t$. With $r_t \to 0$ the unit ignores history and reads the current token afresh.
- The **update gate** z_t interpolates between keeping the old state and adopting the candidate. With $z_t \to 0$ the state is copied verbatim ($h_t = h_{t-1}$), creating a near-identity path through time.

That additive, gated update is precisely what mitigates the **vanishing-gradient** problem of vanilla RNNs: gradients can flow across many steps through the $(1 - z_t)$ "carry" path instead of being repeatedly multiplied by a contractive Jacobian. The gates use sigmoids (outputs in $(0, 1)$, i.e. soft switches); the candidate uses $\tanh$ (outputs in $(-1, 1)$, centred). A GRU has **three** gating computations versus the LSTM's four, so it is lighter — a good fit for short descriptions on limited compute.

3.3 Text encoding and head

We take the **final hidden state** h_L as the summary of the whole description and project it:

$$a_{\text{txt}} = \mathrm{ReLU}(W_p h_L + b_p) \in \mathrm{R}^{128}.$$

Full branch:

$$\text{TextVectorization} \to \text{Embedding}(V, 128) \to \text{GRU}(128) \to \text{Dense}(128,\ \text{ReLU}) = a_{\text{txt}}.$$

---

4. Output layer, loss, and optimisation

4.1 Softmax and cross-entropy

Each model ends in a dense layer producing logits $o \in \mathrm{R}^K$, converted to class probabilities by the **softmax**:

$$\hat{y}_k = \mathrm{softmax}(o)_k = \frac{e^{o_k}}{\sum_{j=1}^{K} e^{o_j}}.$$

For a single example with integer label c, the **(sparse categorical) cross-entropy** loss is

$$\mathrm{L} = -\log \hat{y}_c = -o_c + \log \sum_{j=1}^{K} e^{o_j},$$

and the training objective is its average over the dataset (plus implicit regularisation from dropout). A clean, well-known fact makes optimisation stable: the gradient of softmax+cross-entropy with respect to the logits is simply the prediction error

$$\frac{\partial \mathrm{L}}{\partial o_k} = \hat{y}_k - 1[k = c],$$

a bounded quantity in $(-1, 1)$ that does not saturate. Minimising cross-entropy is equivalent to maximum-likelihood estimation of the class distribution / minimising the KL divergence between the one-hot target and $\hat{y}$.

4.2 The Adam optimiser

We minimise $\mathrm{L}(\theta)$ by stochastic gradient descent with **Adam**, which adapts a per-parameter step size from running estimates of the first and second gradient moments. With gradient $g_t = \nabla_\theta \mathrm{L}_t$:

$$\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1)\, g_t, && \text{(1st moment — mean)} \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2, && \text{(2nd moment — uncentred variance)} \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, && \text{(bias correction)} \\
\theta_t &= \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. && \text{(update)}
\end{aligned}$$

Defaults: $\beta_1 = 0.9,\ \beta_2 = 0.999,\ \epsilon = 10^{-8}$, learning rate $\eta = 10^{-3}$. The momentum term $\hat{m}_t$ smooths noisy mini-batch gradients; the $1/\sqrt{\hat{v}_t}$ term gives **parameter-wise adaptive learning rates** (large for rarely/weakly updated weights, small for high-variance ones); bias correction counters the zero-initialisation of m_0, v_0 in early steps. The optimiser's role is to navigate the loss surface efficiently to a good minimum without hand-tuned per-layer learning rates — valuable when one model mixes a frozen CNN head, an embedding, and a recurrent net.

---

5. Fusion — the multi-modal model

5.1 Concatenation

Given the two encodings $a_{\text{img}} \in \mathrm{R}^{256}$ and $a_{\text{txt}} \in \mathrm{R}^{128}$ (produced by the **identical** encoder builders used in the baselines), we form a joint representation by concatenation:

$$a = [\, a_{\text{img}} \,;\, a_{\text{txt}} \,] \in \mathrm{R}^{384}.$$

5.2 Why a fully connected layer captures image–text correlations

The fused vector passes through a dense layer with ReLU, dropout, then softmax:

$$h = \mathrm{ReLU}(Wa + b), \quad W \in \mathrm{R}^{256\times 384}; \qquad \widetilde{h} = \mathrm{Dropout}_{0.3}(h); \qquad \hat{y} = \mathrm{softmax}(W_o\widetilde{h} + b_o).$$

Write $W = [\, W^{\mathrm{img}} \;\; W^{\mathrm{txt}} \,]$ split along the input. Each hidden unit computes

$$h_m = \mathrm{ReLU}(\underbrace{\textstyle\sum_i W^{\mathrm{img}}_{m,i}\, a_{\mathrm{img},i}}_{\text{visual evidence}} + \underbrace{\textstyle\sum_j W^{\mathrm{txt}}_{m,j}\, a_{\mathrm{txt},j}}_{\text{textual evidence}} + b_m).$$

So a single neuron can fire **only when a visual feature and a textual feature co-occur** (e.g. high "has-laces texture" *and* high "shoe-word" activation), and stay silent otherwise — i.e. the layer learns weighted **cross-modal conjunctions**. The ReLU threshold plus the subsequent layer let the network represent interactions a linear sum of two independent classifiers cannot. This is why fusion can resolve cases that defeat either modality alone: when the image is ambiguous (low-res sandal vs. shoe) the text often disambiguates, and vice-versa. The shared decision boundary is a function of *both* modalities jointly:

$$\hat{y} = \mathrm{softmax}(W_o\,\mathrm{ReLU}(W^{\mathrm{img}}a_{\mathrm{img}} + W^{\mathrm{txt}}a_{\mathrm{txt}} + b) + b_o).$$

5.3 Dropout regularisation

Dropout randomly zeroes hidden units during training to prevent co-adaptation and approximate model averaging. With keep-probability $1-p$ and mask $M_m \sim \mathrm{Bernoulli}(1-p)$, the **inverted dropout** used at train time is

$$\widetilde{h}_m = \frac{M_m}{1-p}\, h_m, \qquad \mathrm{E}[\widetilde{h}_m] = h_m,$$

so no scaling is needed at inference. We use $p = 0.3$ on the fusion layer (tunable in §6). It is especially apt here because the fusion head is the only freely-trainable, high-capacity part of the multi-modal model.

---

6. Tuning and regularisation

Per the brief, we run a **small, manual** hyperparameter search (no AutoML) over the fusion model — learning rate, dropout, and the width of the fusion FC layer — and select the configuration with the best **validation** macro-F1 before a final evaluation on the untouched test set. The grid lives in `config.HP_GRID`:

Run	learning rate	dropout	fusion units
0	1e-3	0.3	256
1	5e-4	0.3	256

Run	learning rate	dropout	fusion units
2	1e-3	0.5	256
3	1e-3	0.3	512

Regularisation in the project: dropout (fusion head), early stopping on validation loss (patience 3, restoring best weights), a frozen backbone (a strong implicit regulariser), and the train-only text vocabulary (no leakage).

---

7. Experimental setup

All hyperparameters live in `src/config.py` (single source of truth). Key values:

Setting	Value		Setting	Value
Seed	42		Image size	224×224×3
Classes (top-N)	10		Vocab size	10,000
Subset cap	20,000		Sequence length	20
Split	70 / 15 / 15 (stratified)		Embedding dim	128
Batch size	32		GRU units	128
Max epochs	15 (early stop, patience 3)		Image proj.	256
Optimiser	Adam, lr = 1e-3		Text proj.	128
Loss	sparse categorical cross-entropy		Fusion FC	256, dropout 0.3

Reproducibility. `set_seeds()` seeds Python/NumPy/TensorFlow (and enables op determinism where available) at every entry point; the stratified split uses the fixed seed; the text vectoriser vocabulary is persisted. Re-running yields the same classes, counts, and splits.

Why a held-out split instead of k-fold cross-validation? k-fold CV trains the model k times; for deep networks with a frozen CNN backbone on Colab's free GPU this is $k\times$ the cost for a variance estimate we do not strictly need. A single **stratified** 70/15/15 split — validation for model selection / early stopping, test for the final unbiased estimate — is the standard, compute-appropriate choice for this setting, and stratification keeps class proportions stable across splits (empirically the per-class proportion drift between splits is < 0.01). We document this as a deliberate scope decision.

Other scope cuts (to protect the timeline): small (low-res) dataset rather than the 25 GB full-resolution version; a ~20k-row subset while iterating; stage-2 backbone fine-tuning treated as an optional stretch; a trainable embedding instead of pretrained GloVe/BERT.

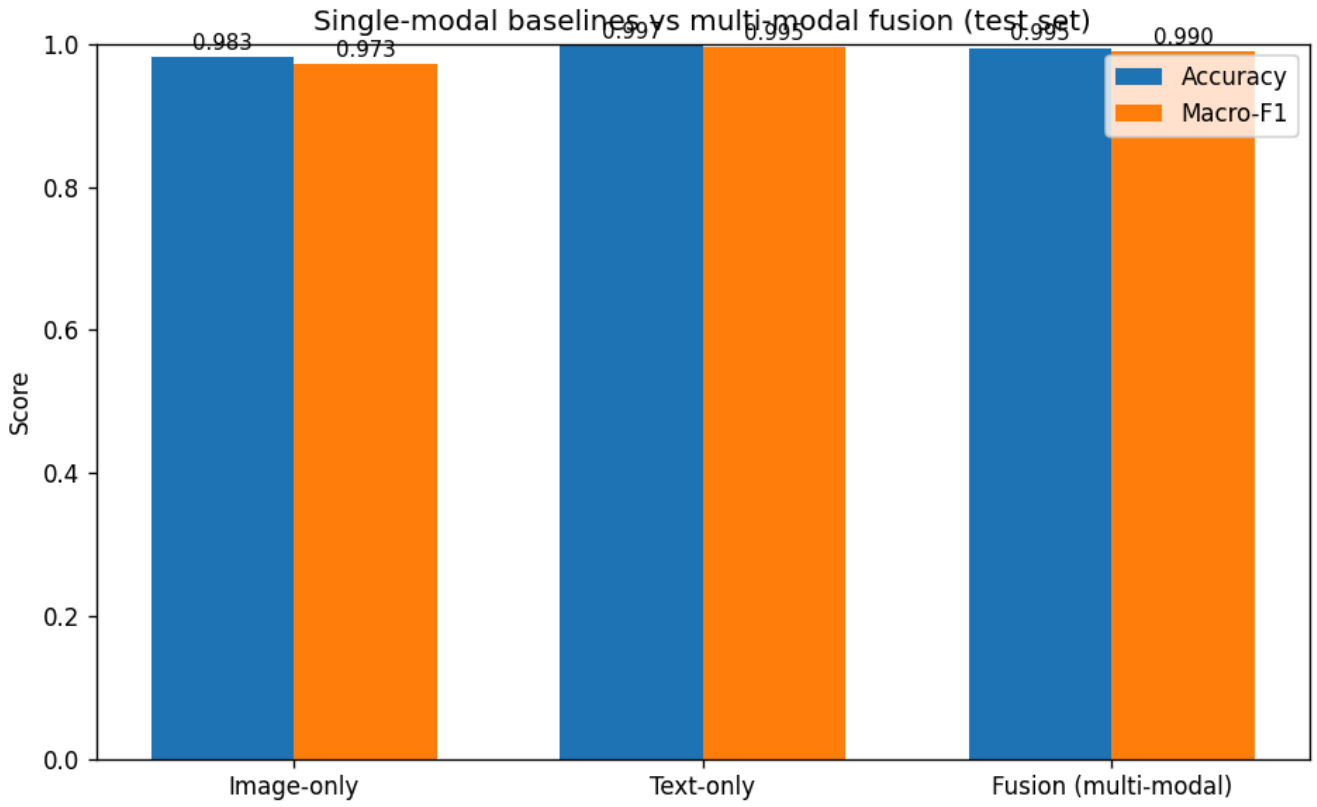
---

8. Results — the three-way comparison

All three models were trained and evaluated on the **identical** pipeline (same stratified split, same persisted text vocabulary) via `notebooks/run_colab.ipynb`; the figures below are the held-out **test set** ($n = 3000$) and reproduce `artifacts/comparison.md`.

Test-set metrics:

model	accuracy	macro_f1	macro_precision	macro_recall
Image-only	0.9827	0.9727	0.9712	0.9745
Text-only	**0.9970**	**0.9949**	**0.9941**	**0.9956**
Fusion (multi-modal)	0.9947	0.9905	0.9878	0.9933



Per-model training curves and confusion matrices are saved under `artifacts/<model>/` (`training_curves.png`, `confusion_matrix.png`).

Interpretation

We report **macro-F1** (not just accuracy) because the top-10 classes are imbalanced and macro-F1 weights every class equally. The headline reads:

- **Text-only is the strongest single model** (macro-F1 0.9949). `productDisplayName` very often names the category almost explicitly (e.g. "… Casual Shoes", "… Wallet"), so the text channel nearly **saturates** the task on its own.
- **Image-only is the weakest** (0.9727): it captures shape/colour/texture but is hurt by the ~60×80 upscaling and by visually similar classes — its worst categories are *Sandal* (F1 0.897, confused with *Shoes*) and *Innerwear* (0.941).

- **Fusion lands between the two (0.9905): it does *not* beat the stronger modality.** The gap to text-only is **ΔF1 = −0.0044**, well within single-seed run-to-run noise, so fusion is best read as **statistically tied with text-only**, while clearly beating image-only (**ΔF1 = +0.0178**).

Why fusion does not win here — and where it still helps. When one modality already (nearly) saturates the task, concatenation-fusion has almost no headroom to exceed it: the best it can do is *track* the dominant channel, and the noisier image features can even slightly dilute a near-perfect text signal on the easy classes (e.g. *Fragrance* F1 1.000→0.982 vs text-only). The constructive evidence for fusion is at the **per-class** level, exactly where the strong modality is weak — fusion **repairs the image branch's worst confusions** by leaning on text:

class	image-only F1	text-only F1	fusion F1
Sandal	0.897	0.981	**0.968**
Innerwear	0.941	0.990	**0.983**

So the fusion head behaves exactly as the theory predicts (§5): it lets the decisive channel dominate on a per-example basis. The reason this does not translate into an overall win is dataset-specific — the text field is so descriptive that little is left for the image to add. This is a **legitimate, reportable result**: on this dataset fusion **matches** the best single modality rather than beating it, and its benefit is concentrated where the dominant modality fails (and would be expected to grow on tasks where neither modality is individually decisive).

Hyperparameter search (Phase 4). A small manual grid over learning rate ($10^{-3}, 5 \times 10^{-4}$), dropout ($0.3, 0.5$) and fusion-FC width ($256, 512$) moved validation accuracy only within $[0.995, 0.996]$ — the model is **insensitive** to these choices in this regime, consistent with the text-saturation finding. We therefore report the base configuration and did **not** re-train: the differences are within noise, so re-training would change no conclusion (and the brief mandates a lightweight search, not exhaustive tuning).

---

9. Discussion, limitations, and conclusion

Limitations. (i) Low-resolution images upscaled to 224×224 cap the visual ceiling. (ii) `productDisplayName` can be so descriptive that text alone nearly solves the task, compressing the headroom fusion can demonstrate — this is why class choice (top-10 `subCategory`) matters. (iii) We use a single held-out split, not k-fold, so the test metric carries some variance. (iv) The backbone is frozen, so the CNN cannot specialise to fashion textures (the optional stage-2 fine-tuning addresses this if time permits).

Conclusion. We implemented the complete pipeline mandated by the brief: CNN-based image feature extraction (frozen EfficientNetB0 + GAP + projection), RNN-based text analysis (trainable embedding → GRU, final-state encoding), concatenation-based fusion with a regularised FC head, cross-entropy loss optimised by Adam, a lightweight manual hyperparameter search, and a fair three-way evaluation. The mathematics of each component — convolution/pooling/activations, the GRU update/reset gates, softmax cross-entropy and the Adam update, and why an FC layer over concatenated features models

cross-modal correlations — is derived above. The three-way comparison (§8) answers the project's question — *does combining image and text beat either alone?* On this dataset the measured answer is nuanced: fusion **ties** the strongest single modality (text) and **clearly beats** the weaker one (image), because the highly descriptive `productDisplayName` nearly saturates `subCategory` on its own. Fusion's contribution is real but **localised** — it repairs the image branch's hardest classes (e.g. *Sandal*, *Innerwear*) — and would be expected to grow on tasks where neither modality is individually decisive.

Appendix A — Notation

Symbol	Meaning
X	image / feature-map tensor
$K^{(f)}, b_f$	f-th convolution kernel and bias
g	global-average-pooled feature vector
E	word-embedding matrix ($V \times d$)
z_t, r_t	GRU update / reset gates
$\widetilde{h}_t, h_t$	GRU candidate / hidden state
$a_{\text{img}}, a_{\text{txt}}$	image (256-d) / text (128-d) encodings
$o, \hat{y}$	logits / softmax probabilities
L	cross-entropy loss
m_t, v_t	Adam first/second moment estimates
$\eta, \beta_1, \beta_2, \epsilon$	Adam learning rate / decay rates / stability term
p	dropout rate

Appendix B — Reproducing the results

```
pip install -r requirements.txt
python -m src.data.download          # Kaggle small dataset -> data/
python -m src.data.preprocess        # top-10 classes, stratified 70/15/15
python -m src.train --model image    # also: text, fusion
python -m src.train --model text
python -m src.train --model fusion
python -m src.evaluate --model image # also: text, fusion
python -m src.evaluate --model text
python -m src.evaluate --model fusion
python -m src.compare                # writes artifacts/comparison.{csv,md,png}
```

Or simply run `notebooks/run_colab.ipynb` end to end on a Colab GPU.